

쌍거울의 무덤 — 팀원 롤 기술서

과거를 비추는 두 청동 거울로 무덤을 빠져나가는 1인칭 웹 퍼즐 게임 (3인 팀)

1. 팀 구성

이름	담당 역할
김두희	기획 · 개발 · 아트 · AI 활용 (공동)
김수곤	기획 · 개발 · 아트 · AI 활용 (공동)
박철오	기획 · 개발 · 아트 · AI 활용 (공동)

저희 팀은 영역을 사람별로 고정해 나누지 않았습니다. 아래 2장의 구현 영역을 기능 단위로 그때그때 나누어 맡았고, 세 명 모두 기획, 개발, 아트 작업에 함께 참여했습니다. 따라서 이 문서는 「누가 무엇을 했는지」 대신 「팀이 실제로 만든 것」을 영역별로 정리합니다.

2. 실제 구현 영역

2-1. 게임 기획 · 퍼즐 설계

- 시간 규칙 설계:** 두 거울이 서로 다른 두 과거를 비추고, 과거에서는 거울 빛이 닿는 곳까지만 행동할 수 있으며, 물건은 거울을 통과하지 못하고 보관 장소를 거쳐야만 시간을 건넌다는 핵심 규칙을 세웠습니다. 과거에서 바꾼 것은 현재에 남지만 그 반대는 없다는 한 방향 원칙도 여기서 정했습니다.
- 세 가지 수수께끼 설계:** ① 돌무더기에 묻힌 열쇠를 과거에서 벽 틈에 숨겨 현재로 가져오는 문제, ② 더 먼 과거의 도구를 가까운 과거로 보내 벽을 열고, 그 결과를 현재에서 거두는 문제, ③ 보물을 벽화에 대어 보고 얻은 세 글자로 가짜 문을 여는 마지막 문제를 차례로 설계했습니다.
- 설계 원칙:** 인위적인 잠금이나 순서 강제를 두지 않고, 무엇이든 시도해 보면서 스스로 규칙을 깨닫게 하는 방향을 지켰습니다. 힌트는 안내문 대신 장면 안에 두었습니다(무너진 자리 위 천장의 금, 시대에 따라 깨져 있는 석판 등).

2-2. 시간 논리 엔진 · 자동 검증 (개발)

- 인과 엔진:** 게임 전체를 하나의 상태로 관리하고, 그 상태에서 세 시대(과거 둘, 현재)의 화면을 매번 다시 계산해서 그려 냅니다. 과거에서 한 행동이 현재에 자동으로 반영되는 것은 이 구조 덕분입니다.
- 막힘 검사:** 플레이어가 도달할 수 있는 모든 진행 상태(112가지)를 자동으로 탐색해서, 어떤 순서로 행동해도 끝까지 풀 수 있음을 기계적으로 확인했습니다. 이 검사로 실제 설계 결함 두 건을 찾아 고쳤습니다.
- 기하 검증:** 거울 빛이 열쇠·벽돌 같은 목표 지점에 실제로 닿는지, 상호작용 지점에 충분한 여유가 있는지, 한 각도에서 두 목표가 동시에 잡혀 문제가 쉬워지지 않는지를 수치로 검사하는 도구를 만들었습니다.
- 테스트 일원화:** 위 검사 전부를 명령 한 번(npm test)으로 돌려, 누구의 변경이든 규칙을 깨면 바로 드러나게 했습니다.

2-3. 3D 그래픽 · 아트

- **장면 제작:** Three.js로 두 방과 세 시대의 장면을 만들고, 재질에는 Poly Haven의 무료(CC0) PBR 텍스처 7종을 썼습니다 — 사암 벽돌(벽), 혼합 석재 타일(바닥), 화강암(문·좌대·제단), 마른 바위(붕괴 잔해), 모래, 회반죽 2종(붕괴 벽·갈라진 천장). 3D 모델은 Poly Pizza의 해골(skull003, Jake K-H)과 1인칭 손(Fists, Sean Tarrant) 두 점을 CC-BY 조건에 맞게 출처를 표기하고 사용했습니다. 거울은 비칠 장면을 실시간으로 따로 그려서 보여 줍니다.
- **그림 아트:** 벽화·가짜 문·석판·다이얼은 먼저 에셋 없이 디자인 초안을 잡은 뒤, 기존 에셋이 어울리는 것은 에셋을 쓰고 나머지는 코드로 직접 그렸습니다. 아누비스 벽화, 상형문자 띠, 별 천장, 글자 다이얼이 코드로 그린 것입니다. 새김 무늬는 흑백 명암으로 먼저 그리고(밝은 곳은 튀어나오고 어두운 곳은 파인 것으로 약속) 이 명암 이미지를 조명 계산에 쓰는 표면 방향 정보, 곧 노멀 맵으로 변환했습니다. 그래서 손전등을 비스듬히 비추면 새김 무늬에 실제로 그늘이 집니다.
- **조명:** 현재는 손전등 하나, 과거는 거울에서 나오는 빛기둥 하나가 유일한 조명입니다. 빛기둥은 벽과 문틈 모양에 맞춰 잘리며, 이 빛의 모양이 곧 이동 가능한 범위라는 규칙을 눈으로 보여 줍니다.

2-4. 연출 · 사운드

- **연출:** 시작하면 윗층 복도를 걷다 바닥이 무너져 떨어지는 도입 장면이, 가짜 문을 열면 물러나는 돌문과 탈출 복도를 걸어 나가는 마무리 장면이 나옵니다. 빈손으로 나가는지, 보물을 들고 나가는지에 따라 마지막 장면이 달라집니다.
- **사운드:** 발소리·돌문·시대별 배경음 등은 OpenGameArt의 무료 음원을 썼습니다(대부분 CC0, 획득 차임·이동 반짝임 등 일부는 CC-BY 3.0으로 출처 표기). 딱 맞는 음원이 없던 소리는 음원을 변형해 만들었습니다 — 예를 들어 무거운 청동 거울이 돌받침 위에서 갈리는 소리는 돌 부딪는 음원을 4분의 1 속도로 늦추고 낮은 합성음을 겹친 것입니다. 음원을 아직 불러오지 못했거나 불러오기에 실패한 환경에서는 브라우저 안에서 합성한 소리가 자동으로 대신합니다.

2-5. 상호작용 · 화면 표시(UI)

- **조준 보조(aim assist):** 멀티 레이캐스트(multi-raycast) 방식입니다. 화면 중앙 한 점이 아니라 그 주변까지 광선 아홉 갈래를 쏘아 검사하고, 중앙에 가까운 것에 가산점을 주어 고릅니다. 덕분에 작은 물체도 정밀하게 겨누지 않아도 잡힙니다.
- **소지품 규칙:** 물건 여러 개를 지니고 숫자 키로 손에 질 것을 고르며, 어느 시대에서든 같은 방식으로 내려놓고 다시 집을 수 있습니다.
- **안내:** 바라보는 대상에 맞는 행동 문구가 뜨고, 힘이 드는 행동(벽돌 당기기)은 길게 누르는 동안 게이지로 진행을 보여 줍니다.

2-6. 최적화 · 품질 관리

- **속도:** 이 게임에서 가장 무거운 작업은 거울입니다. 거울 하나를 보여 주려면 유리에 비칠 과거 장면을 매번 한 번 더 그려야 하는데, 거울이 넷(각 방의 거울과, 과거에서 현재를 되비추는 거울)이라 그대로 두면 장면을 다섯 번 그리는 셈이 됩니다. 그래서 플레이어가 바라보고 있는 거울만 매 화면 온전한 화질로 그리고, 시야 가장자리나 멀리 있는 거울은 그리는 횟수와 화질을 낮췄습니다. 여기에 그림자 계산의 해상도를 줄이고, 첫 클릭 순간에 필요한 그

래픽 자원을 미리 준비해 두어(그래야 처음 거울을 볼 때 화면이 멈칫하지 않습니다), 내장 그래픽에서도 매끄럽게 돌아갑니다.

- **진단:** 게임이 뜨지 않는 문제를 조사할 수 있도록, 불러오기 오류가 나면 시작 화면에 원인이 그대로 표시되게 했습니다. 화면 없는 브라우저로 게임을 자동으로 켜고 클릭해 보는 검사도 만들었습니다.
- **배포:** 저장소에 반영하면 자동으로 웹에 배포됩니다. 설치 없이 주소만으로 플레이할 수 있습니다.

3. 협업 · 분업 방식

- **기능 단위 분담:** 사람마다 영역을 고정하지 않고, 그날 필요한 기능을 나눠 맡았습니다. 같은 파일을 여러 명이 만지게 되므로, 아래의 통합 규칙으로 충돌을 관리했습니다.
- **하나의 저장소, 작은 커밋:** 모든 작업을 git 저장소 하나에 모으고, 변경을 작게 나눠 자주 반영했습니다. 큰 변경(마지막 문 퍼즐)은 따로 가지에서 만들어 검증을 마친 뒤 합쳤습니다.
- **자동 검사로 서로를 보호:** 병행 작업에서 상대의 변경이 게임 규칙을 깨지 않았는지를 사람이 아니라 검사 도구가 확인합니다. 합칠 때마다 전체 검사를 돌리는 것을 규칙으로 삼았습니다.
- **상호 검증:** 한 사람이 만든 것은 다른 사람이 수치로 재거나 직접 플레이해서 확인한 뒤 반영했습니다. 겉보기 문제(떠 보이는 물건, 기울어진 거울 등)는 플레이한 사람이 바로 보고하고 그 자리에서 고쳤습니다.
- **AI 도구 활용:** Claude Code를 코딩 에이전트로 활용했습니다. 검증 도구 작성, 거울 반사 각도 같은 기하 계산, 버그 원인 추적, 문서 작성을 맡겼고, 설계 판단과 최종 확인은 팀원이 직접 했습니다.

프로젝트 저장소: <https://github.com/ppco915/mirrorlight> · 플레이: <https://ppco915.github.io/mirrorlight/>